

Assignment 6 - Radix-2 8-Point FFT (Streaming, Pipelined) Implementation

Sai Akhila Reddy Turpu
EE24BTECH11055

1 Introduction

The Fast Fourier Transform (FFT) is an efficient algorithm used to compute the Discrete Fourier Transform (DFT). The DFT converts a discrete-time signal from the time domain into its frequency domain representation.

The N -point DFT is given by:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j\frac{2\pi}{N}kn} \quad (1)$$

Direct computation requires $O(N^2)$ operations, whereas FFT reduces this to $O(N \log N)$.

2 Radix-2 FFT Algorithms

2.1 Decimation-In-Time (DIT)

- Input is split into even and odd indexed samples.
- Recursive decomposition is performed first.
- Twiddle factors are applied after smaller FFTs.

2.2 Decimation-In-Frequency (DIF)

- Output is split into even and odd indices.
- Butterfly operations are performed first.
- Twiddle factors are applied before recursion.

3 Chosen Approach: DIF FFT

In this work, the Decimation-In-Frequency (DIF) approach is used due to its suitability for streaming architectures.

For an 8-point FFT, the computation is divided into $\log_2(8) = 3$ stages.

3.1 Butterfly Operation

Each butterfly computes:

$$A = x_0 + x_1 \tag{2}$$

$$B = (x_0 - x_1) \cdot W_N^k \tag{3}$$

where $W_N^k = e^{-j\frac{2\pi k}{N}}$ is the twiddle factor.

4 Streaming Architecture

Streaming input means:

- One complex sample (8-bit real + 8-bit imaginary) arrives per clock cycle.
- 8 cycles are required to receive all inputs.
- Pipeline registers are used between stages.

The pipeline consists of 3 stages:

- Stage 1: Distance = 4
- Stage 2: Distance = 2
- Stage 3: Distance = 1

5 SDF Stage-1 (DIF FFT) Architecture Explanation

The given Verilog module implements the first stage of a radix-2 Decimation-In-Frequency (DIF) FFT using a Single-Path Delay Feedback (SDF) architecture.

5.1 Overview

The design processes one complex input sample per clock cycle:

$$x[n] = \text{in_r} + j \text{in_i}$$

It produces outputs:

$$y[n] = \text{out_r} + j \text{out_i}$$

The architecture consists of:

- A delay line of length 4
- Butterfly computation (add/subtract)
- Twiddle factor multiplication

5.2 Delay Line Operation (First 4 Cycles)

For the first 4 clock cycles:

$$n = 0, 1, 2, 3$$

The input samples are stored in a shift register:

$$\begin{aligned} \text{delay}[0] &\leftarrow x[n] \\ \text{delay}[1] &\leftarrow \text{delay}[0] \\ \text{delay}[2] &\leftarrow \text{delay}[1] \\ \text{delay}[3] &\leftarrow \text{delay}[2] \end{aligned}$$

Thus after 4 cycles:

$$\text{delay}[3] = x[0], \quad \text{delay}[2] = x[1], \quad \text{delay}[1] = x[2], \quad \text{delay}[0] = x[3]$$

During this phase, the output is simply the delayed version:

$$y[n] = \text{delay}[3]$$

However, these outputs are not yet valid for FFT computation.

5.3 Butterfly Computation (From Cycle 4 Onwards)

Starting from:

$$n = 4$$

Each new input is combined with the delayed sample:

$$x[n] \quad \text{and} \quad x[n-4]$$

The radix-2 DIF butterfly is computed as:

5.3.1 Addition (Upper Branch Output)

$$y[n] = x[n] + x[n-4]$$

This is directly sent to the output:

$$\text{out_r} = \text{in_r} + \text{delay_r}[3]$$

$$\text{out_i} = \text{in_i} + \text{delay_i}[3]$$

5.3.2 Subtraction (Lower Branch)

$$d[n] = x[n-4] - x[n]$$

5.4 Twiddle Factor Multiplication

The difference signal is multiplied by a twiddle factor:

$$W_8^k = \cos \theta - j \sin \theta$$

where:

$$k = n \bmod 4$$

The complex multiplication is:

$$\begin{aligned} d'_r &= (d_r \cdot W_r) - (d_i \cdot W_i) \\ d'_i &= (d_r \cdot W_i) + (d_i \cdot W_r) \end{aligned}$$

In fixed-point implementation (Q format), scaling is applied:

$$d_r^{\text{scaled}} = \frac{d'_r}{2^5}, \quad d_i^{\text{scaled}} = \frac{d'_i}{2^5}$$

5.5 Feedback Mechanism

The twiddle-multiplied result is fed back into the delay line:

$$\text{delay}[0] \leftarrow d[n] \cdot W_8^k$$

and shifted forward:

$$\text{delay}[i] \leftarrow \text{delay}[i - 1], \quad i = 1, 2, 3$$

5.6 Processing Sequence for 8-Point FFT

- Cycles 0–3:
 - Inputs $x[0]$ to $x[3]$ are stored
 - No butterfly operation yet
- Cycles 4–7:
 - Butterfly starts:

$$(x[4], x[0]), (x[5], x[1]), (x[6], x[2]), (x[7], x[3])$$
 - Outputs:

$$x[n] + x[n - 4]$$
 - Differences are twiddle-multiplied and stored
- Final cycles:
 - Stored (twiddle-processed) values propagate through delay line
 - These correspond to the lower branch outputs of DIF FFT

5.7 Key Characteristics of DIF SDF

- Input is in natural order
- Butterfly uses:

$$x[n] \pm x[n-4]$$

- Twiddle multiplication occurs **after subtraction**
- Uses single delay line (efficient hardware)
- Outputs are produced in a streamed fashion

6 MATLAB Verification Code

```
clear; clc;

N = 8;
fractional_bits = 5;
scale_factor = 2^fractional_bits;

% rectangular pulse
x_real = [1, 1, 0, 0, 0, 1, 0, 1];
x_imag = zeros(1, N);
x = complex(x_real, x_imag);

x_fixed = round(x * scale_factor);

fft_matlab = fft(x_fixed);

br_indices = bitrevorder(0:N-1) + 1;
expected_out = fft_matlab(br_indices);

fileID = fopen('fft_output.txt','w');
fprintf(fileID, 'Cycle Bin Real Imag\n');

fprintf('Input Data \n');
for i = 1:N
    fprintf('send_sample(16''sd%d, 16''sd%d); // x[%d]\n', real(x_fixed(i)), imag(x_fixed(i)), i);
end

fprintf('Expected Bit-Reversed Output\n');
fprintf('Cycle | Bin | Real (Int) | Imag (Int) | Hex Equiv\n');

bins = [0, 4, 2, 6, 1, 5, 3, 7];
```

```

for i = 1:N
    real_val = round(real(expected_out(i)));
    imag_val = round(imag(expected_out(i)));

    fprintf(' %d | X(%d) | %d | %d | 16'h%s\n', ...
        i, bins(i), real_val, imag_val, ...
        dec2hex(typecast(int16(real_val), 'uint16'), 4));

    fprintf(fileID, '%d %d %d %d\n', ...
        i, bins(i), real_val, imag_val);
end

fclose(fileID);
disp('Data also written to fft_output.txt');

```

7 Verilog Implementation

7.1 Stage 1 Module

```

module sdf_stage1 (
    input clk,
    input rst,
    input signed [7:0] in_r, in_i,
    output reg signed [8:0] out_r, out_i,
    output reg out_valid
);

reg signed [7:0] tw_r, tw_i;

always @(*) begin
    case (count[1:0])
        2'd0: begin tw_r = 8'sd32; tw_i = 8'sd0; end // 1.0
        2'd1: begin tw_r = 8'sd22; tw_i = -8'sd22; end // 0.707
        2'd2: begin tw_r = 8'sd0; tw_i = -8'sd32; end // -j1
        2'd3: begin tw_r = -8'sd22; tw_i = -8'sd22; end // -0.707 - j0.707
    endcase
end

reg signed [8:0] delay_r [0:3];
reg signed [8:0] delay_i [0:3];

```

```

reg signed [16:0] dr;
reg signed [16:0] di;
integer i;
reg [2:0] count;

always @(posedge clk or posedge rst) begin
    if (rst)
    begin
        count <= 0;
        out_valid <=0;
    end
    else
    begin
        count <= count + 1;
        if(count == 4)
            out_valid <= 1;
    end
end

end

always @(posedge clk) begin
//$display("%d %d %d %d %d %d %d %d", count, in_r, out_r,out_i, delay_r[0],delay_r[1]
    if (count < 4)
    begin
        delay_r[0] <= in_r;
        delay_i[0] <= in_i;
        for(i=1;i<4;i=i+1)
        begin
            delay_r[i] <= delay_r[i-1];
            delay_i[i] <= delay_i[i-1];
        end
        out_r <= delay_r[3];
        out_i <= delay_i[3];
    end
    else
    begin
        out_r <= in_r + delay_r[3];
        out_i <= in_i + delay_i[3];
        dr = (((delay_r[3] - in_r) * tw_r) - ((delay_i[3] - in_i) * tw_i))>>>5 ;
        di = (((delay_r[3] - in_r) * tw_i) + ((delay_i[3] - in_i) * tw_r))>>>5 ;
        delay_r[0] <=dr;
        delay_i[0] <=di;
        for(i=1; i<4; i=i+1) begin
            delay_r[i] <= delay_r[i-1];
            delay_i[i] <= delay_i[i-1];

```

```

        end
    end
end

endmodule

```

8 SDF Stage-1 (DIF FFT) Architecture Explanation

The given Verilog module implements the first stage of a radix-2 Decimation-In-Frequency (DIF) FFT using a Single-Path Delay Feedback (SDF) architecture.

8.1 Overview

The design processes one complex input sample per clock cycle:

$$x[n] = \text{in_r} + j \text{ in_i}$$

It produces outputs:

$$y[n] = \text{out_r} + j \text{ out_i}$$

The architecture consists of:

- A delay line of length 4
- Butterfly computation (add/subtract)
- Twiddle factor multiplication

8.2 Delay Line Operation (First 4 Cycles)

For the first 4 clock cycles:

$$n = 0, 1, 2, 3$$

The input samples are stored in a shift register:

$$\begin{aligned}
 \text{delay}[0] &\leftarrow x[n] \\
 \text{delay}[1] &\leftarrow \text{delay}[0] \\
 \text{delay}[2] &\leftarrow \text{delay}[1] \\
 \text{delay}[3] &\leftarrow \text{delay}[2]
 \end{aligned}$$

Thus after 4 cycles:

$$\text{delay}[3] = x[0], \quad \text{delay}[2] = x[1], \quad \text{delay}[1] = x[2], \quad \text{delay}[0] = x[3]$$

During this phase, the output is simply the delayed version:

$$y[n] = \text{delay}[3]$$

However, these outputs are not yet valid for FFT computation.

8.3 Butterfly Computation (From Cycle 4 Onwards)

Starting from:

$$n = 4$$

Each new input is combined with the delayed sample:

$$x[n] \quad \text{and} \quad x[n-4]$$

The radix-2 DIF butterfly is computed as:

8.3.1 Addition (Upper Branch Output)

$$y[n] = x[n] + x[n-4]$$

This is directly sent to the output:

$$\text{out_r} = \text{in_r} + \text{delay_r}[3]$$

$$\text{out_i} = \text{in_i} + \text{delay_i}[3]$$

8.3.2 Subtraction (Lower Branch)

$$d[n] = x[n-4] - x[n]$$

8.4 Twiddle Factor Multiplication

The difference signal is multiplied by a twiddle factor:

$$W_8^k = \cos \theta - j \sin \theta$$

where:

$$k = n \bmod 4$$

The complex multiplication is:

$$d'_r = (d_r \cdot W_r) - (d_i \cdot W_i)$$

$$d'_i = (d_r \cdot W_i) + (d_i \cdot W_r)$$

In fixed-point implementation (Q format), scaling is applied:

$$d_r^{\text{scaled}} = \frac{d'_r}{2^5}, \quad d_i^{\text{scaled}} = \frac{d'_i}{2^5}$$

8.5 Feedback Mechanism

The twiddle-multiplied result is fed back into the delay line:

$$\text{delay}[0] \leftarrow d[n] \cdot W_8^k$$

and shifted forward:

$$\text{delay}[i] \leftarrow \text{delay}[i-1], \quad i = 1, 2, 3$$

8.6 Processing Sequence for 8-Point FFT

- Cycles 0–3:
 - Inputs $x[0]$ to $x[3]$ are stored
 - No butterfly operation yet
- Cycles 4–7:
 - Butterfly starts:
$$(x[4], x[0]), (x[5], x[1]), (x[6], x[2]), (x[7], x[3])$$
 - Outputs:
$$x[n] + x[n - 4]$$
 - Differences are twiddle-multiplied and stored
- Final cycles:
 - Stored (twiddle-processed) values propagate through delay line
 - These correspond to the lower branch outputs of DIF FFT

8.7 Key Characteristics of DIF SDF

- Input is in natural order
- Butterfly uses:
$$x[n] \pm x[n - 4]$$
- Twiddle multiplication occurs **after subtraction**
- Uses single delay line (efficient hardware)
- Outputs are produced in a streamed fashion

8.8 Stage 2 Module

```
module sdf_stage2 (  
    input clk,  
    input rst,  
    input enable,  
    input signed [8:0] in_r, in_i,  
    output reg signed [8:0] out_r, out_i,  
    output reg out_valid  
);  
  
reg signed [15:0] dr;
```

```

reg signed [15:0] di;
reg signed [7:0] tw_r, tw_i;

always @(*) begin
    case (count[0])
        2'd0: begin tw_r = 8'sd32; tw_i = 8'sd0; end // W8^0
        2'd1: begin tw_r = 8'sd0; tw_i = -8'sd32; end // W8^1
    endcase
end

reg signed [8:0] delay_r [0:1];
reg signed [8:0] delay_i [0:1];

integer i;

reg [1:0] count;
always @(posedge clk or posedge rst) begin
    if (rst)
        begin
            count <= 3;
            out_valid <= 0;
        end
    else
        begin
            count <= count + 1;
            if(count == 2 && enable)
                out_valid <= 1;
        end
    end

end

always @(posedge clk) begin

    // $display("%d %d %d %d %d %d %d %d", count, enable, in_r, out_r, out_i, delay_r[0],d
    if(enable) begin
        if (count < 2)
            begin
                delay_r[0] <= in_r;
                delay_i[0] <= in_i;
                for(i=1;i<2;i=i+1)
                    begin
                        delay_r[i] <= delay_r[i-1];
                        delay_i[i] <= delay_i[i-1];
                    end
            end
        end
    end
end

```

```

        out_r <= delay_r[1];
        out_i <= delay_i[1];
    end
    else
    begin
        out_r <= in_r + delay_r[1];
        out_i <= in_i + delay_i[1];

        dr = (((delay_r[1] - in_r) * tw_r) - ((delay_i[1] - in_i) * tw_i))>>>5 ;
        di = (((delay_r[1] - in_r) * tw_i) + ((delay_i[1] - in_i) * tw_r))>>>5 ;
        delay_r[0] <=dr;
        delay_i[0] <=di;

        // delay_r[0] <= delay_r[1] - in_r;
        // delay_i[0] <= delay_i[1] - in_i;

        for(i=1; i<2; i=i+1)
        begin
            delay_r[i] <= delay_r[i-1];
            delay_i[i] <= delay_i[i-1];
        end
    end
end
endmodule

```

9 SDF Stage-2 and Stage-3 (DIF FFT) Explanation

9.1 General Observation Across Stages

In an $N = 8$ point radix-2 DIF FFT using SDF architecture:

- Stage 1 uses delay length = 4
- Stage 2 uses delay length = 2
- Stage 3 uses delay length = 1

In general:

$$\text{Delay length at stage } s = \frac{N}{2^s}$$

Each stage performs:

- Buffering (first half of inputs)

- Butterfly operation (second half)
- Twiddle multiplication on the difference branch

9.2 Stage-2 Operation

9.2.1 Overview

Stage-2 processes the outputs of Stage-1. It uses:

- Delay line of length 2
- Twiddle factors: W_8^0 and W_8^2

The twiddle selection depends on:

$$k = n \bmod 2$$

9.2.2 Twiddle Factors

$$\begin{aligned} W_8^0 &= 1 + j0 \\ W_8^2 &= 0 - j1 \end{aligned}$$

In fixed-point (Q format):

$$(32, 0), \quad (0, -32)$$

9.2.3 First 2 Cycles (Buffering Phase)

For:

$$n = 0, 1$$

Inputs are stored in delay registers:

$$\begin{aligned} \text{delay}[0] &\leftarrow x[n] \\ \text{delay}[1] &\leftarrow \text{delay}[0] \end{aligned}$$

Output:

$$y[n] = \text{delay}[1]$$

No butterfly is performed yet.

9.2.4 Butterfly Phase (From Cycle 2)

For:

$$n \geq 2$$

Butterfly is computed between:

$$x[n] \quad \text{and} \quad x[n-2]$$

Addition (Upper Branch)

$$y[n] = x[n] + x[n - 2]$$

Subtraction (Lower Branch)

$$d[n] = x[n - 2] - x[n]$$

9.2.5 Twiddle Multiplication

$$d'_r = (d_r \cdot W_r) - (d_i \cdot W_i)$$

$$d'_i = (d_r \cdot W_i) + (d_i \cdot W_r)$$

Scaling:

$$d_r^{\text{scaled}} = \frac{d'_r}{2^5}, \quad d_i^{\text{scaled}} = \frac{d'_i}{2^5}$$

9.2.6 Feedback

The twiddle-multiplied result is fed back:

$$\text{delay}[0] \leftarrow d[n] \cdot W_8^k$$

and shifted:

$$\text{delay}[1] \leftarrow \text{delay}[0]$$

9.2.7 Key Insight for Stage-2

- Same structure as Stage-1
- Reduced delay length (2 instead of 4)
- Fewer twiddle factors
- Faster butterfly pairing:

$$(x[2], x[0]), (x[3], x[1]), (x[6], x[4]), (x[7], x[5])$$

9.3 Stage-3 Operation

9.3.1 Overview

Stage-3 is the final stage with:

- Delay length = 1
- No twiddle multiplication (only $W_8^0 = 1$)

9.3.2 First Cycle (Buffering)

For:

$$n = 0$$

$$\text{delay} \leftarrow x[0]$$

Output:

$$y[0] = \text{delay}$$

9.3.3 Butterfly Phase (From Cycle 1)

For:

$$n \geq 1$$

Butterfly is performed between:

$$x[n] \quad \text{and} \quad x[n-1]$$

Addition

$$y[n] = x[n] + x[n-1]$$

Subtraction

$$d[n] = x[n-1] - x[n]$$

9.3.4 Feedback (No Twiddle)

Since:

$$W_8^0 = 1$$

No multiplication is required:

$$\text{delay} \leftarrow x[n-1] - x[n]$$

9.3.5 Key Insight for Stage-3

- Simplest stage
- Only 1 register
- No twiddle multiplication
- Pure butterfly operation
- Final output stage of FFT

9.4 Overall Pipeline Behavior

- Stage-1: Combines samples spaced by 4
- Stage-2: Combines samples spaced by 2
- Stage-3: Combines adjacent samples

This follows the DIF decomposition:

Stride reduces as: $4 \rightarrow 2 \rightarrow 1$

The only differences across stages are:

- Delay length: $4 \rightarrow 2 \rightarrow 1$
- Twiddle complexity: Full $\rightarrow$ Reduced $\rightarrow$ None

9.5 Stage 3 Module

```
module sdf_stage3 (  
    input clk,  
    input rst,  
    input enable,  
    input signed [8:0] in_r, in_i,  
    output reg signed [8:0] out_r, out_i,  
    output reg out_valid  
);  
  
reg signed [8:0] delay_r;  
reg signed [8:0] delay_i;  
  
integer i;  
  
reg count;  
always @(posedge clk or posedge rst) begin  
    if (rst)  
        begin  
            count <= 0;  
            out_valid <= 0;  
        end  
    else  
        begin  
            count <= count + 1;  
            if(count == 1)  
                out_valid <= 1;  
        end  
end
```



```

end

always @(posedge clk) begin
    $display("%d %d %d %d %d %d %d", count, in_r, out_r, out_i, delay_r[0], delay_r[1], out_i);
    if(enable)
        begin
            if (count == 0)
                begin
                    delay_r <= in_r;
                    delay_i <= in_i;
                    out_r <= delay_r;
                    out_i <= delay_i;
                end
            else
                begin
                    out_r <= in_r + delay_r;
                    out_i <= in_i + delay_i;
                    delay_r <= delay_r - in_r;
                    delay_i <= delay_i - in_i;
                end
            end
        end
end

endmodule

```

9.6 Top Module

```

module fft_stage12 (
    input clk, rst,
    input signed [7:0] in_r, in_i,
    output signed [8:0] out_r, out_i,
    output out_valid
);
    wire signed [8:0] s1_r, s1_i, s2_r, s2_i;
    wire v1, v2;

    sdf_stage1 u1 (.clk(clk), .rst(rst), .in_r(in_r), .in_i(in_i),
                  .out_r(s1_r), .out_i(s1_i), .out_valid(v1));

    sdf_stage2 u2 (.clk(clk), .rst(rst), .enable(v1), .in_r(s1_r), .in_i(s1_i),
                  .out_r(s2_r), .out_i(s2_i), .out_valid(v2));

    sdf_stage3 u3 (.clk(clk), .rst(rst), .enable(v2), .in_r(s2_r), .in_i(s2_i),

```

```

                                .out_r(out_r), .out_i(out_i), .out_valid(out_valid));
endmodule

```

10 Control Signals: enable and out_valid

10.1 Purpose

In the SDF FFT pipeline, each stage does not produce valid outputs immediately due to internal buffering. Therefore:

- `out_valid` indicates when a stage's output is valid
- `enable` ensures the next stage processes only valid data

10.2 out_valid Signal

- Generated by each stage
- Becomes HIGH after initial buffering delay
- Indicates that the output data is now meaningful

Stage-wise behavior:

- Stage-1:

$$\text{out_valid} = 1 \quad \text{when count} = 4$$

- Stage-2:

$$\text{out_valid} = 1 \quad \text{when count} = 2 \text{ and enable} = 1$$

- Stage-3:

$$\text{out_valid} = 1 \quad \text{when count} = 1$$

Thus, each stage asserts validity only after its required delay:

$$4 \rightarrow 2 \rightarrow 1$$

10.3 enable Signal

- Input control signal to a stage
- Comes from `out_valid` of the previous stage

Connections in top module:

$$\text{Stage-2 enable} = v1 \text{ (Stage-1 out_valid)}$$

$$\text{Stage-3 enable} = v2 \text{ (Stage-2 out_valid)}$$

11.2 Resource Utilization

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Apr 12 20:48:58 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	FFT
Top-level Entity Name	fft_stage12
Family	Cyclone V
Device	5CGXFC9E7F35C8
Timing Models	Final
Logic utilization (in ALMs)	78 / 113,560 (< 1 %)
Total registers	197
Total pins	37 / 616 (6 %)
Total virtual pins	0
Total block memory bits	0 / 12,492,800 (0 %)
Total DSP Blocks	4 / 342 (1 %)
Total HSSI RX PCSs	0 / 12 (0 %)
Total HSSI PMA RX Deserializers	0 / 12 (0 %)
Total HSSI TX PCSs	0 / 12 (0 %)
Total HSSI PMA TX Serializers	0 / 12 (0 %)
Total PLLs	0 / 20 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 2: Flow summary

12 Output Verification

For the input [1,1,0,0,0,1,0,1], the outputs of MATLAB code and verilog code were compared.

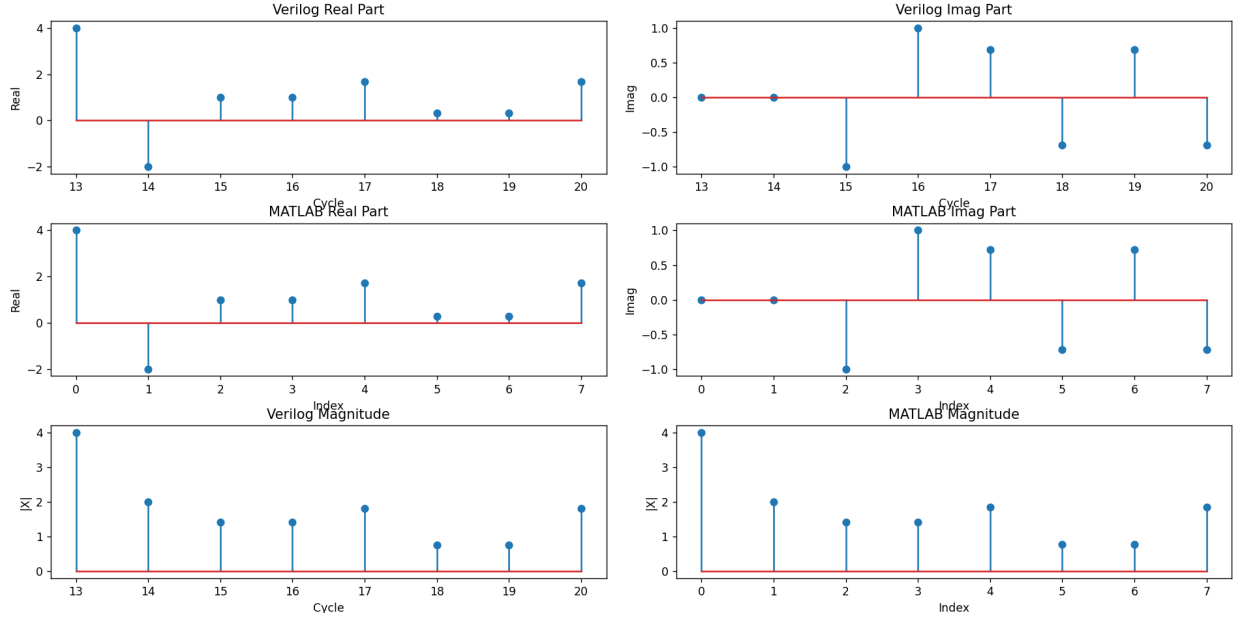


Figure 3: Plot of verilog v/s MATLAB

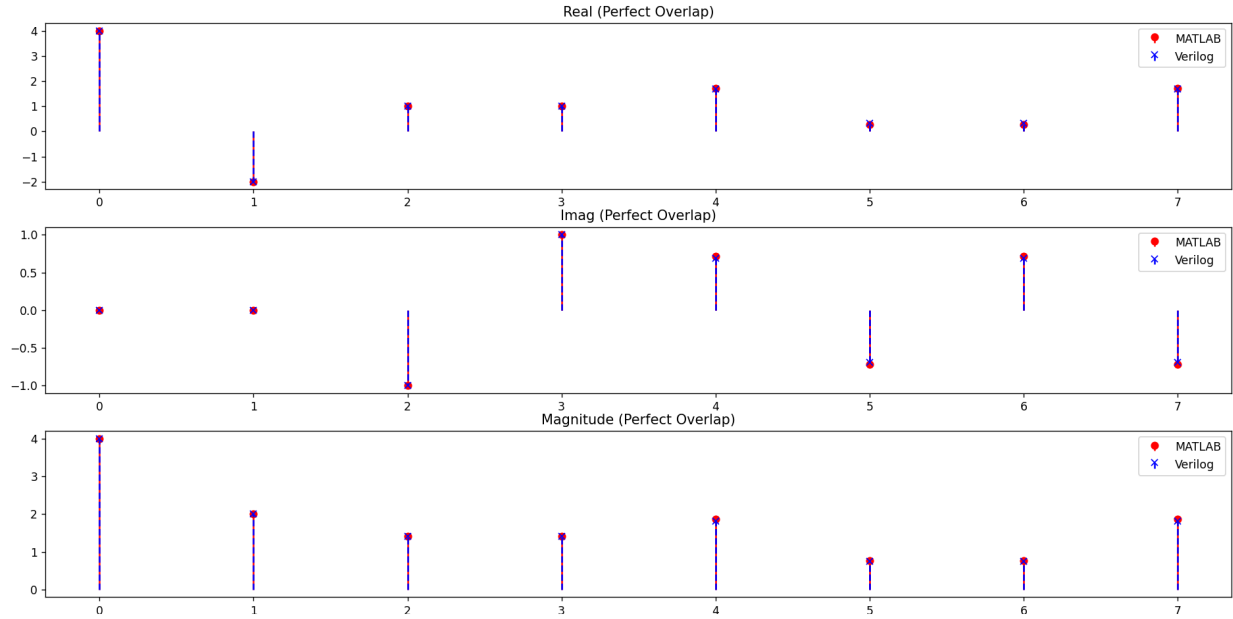


Figure 4: Plot of verilog v/s MATLAB(Overlap)

As we can see, both of them overlapped perfectly, verifying the correctness of the verilog code implemented.

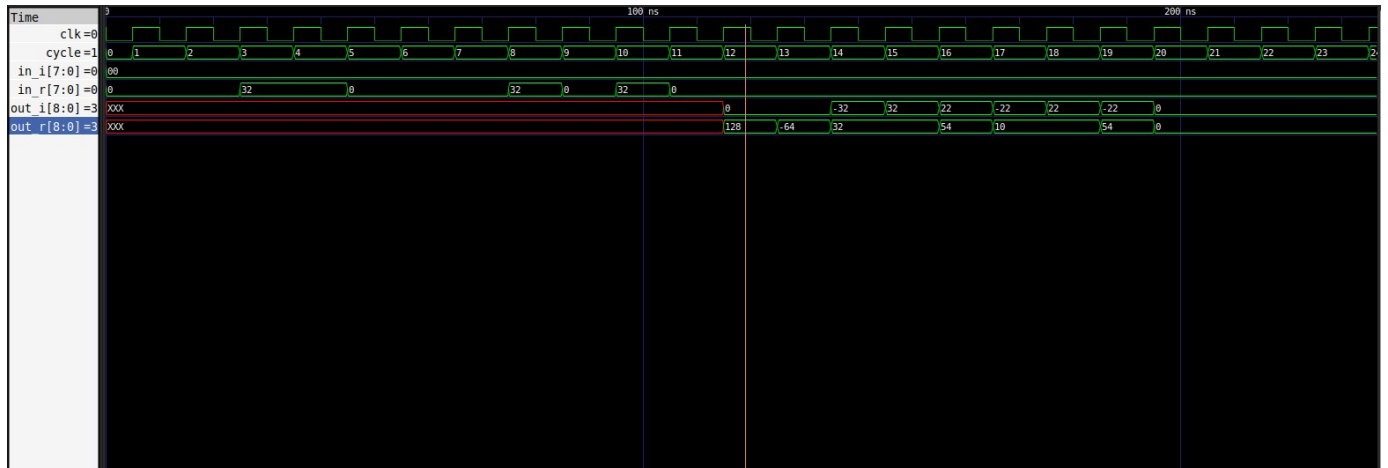


Figure 5: GTKwave

13 Intel IP Comparison

- With length of FFT being 64, but streaming input Timing analysis

Slow 1100mV 85C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	87.45 MHz	87.45 MHz	altera...ed_tck	
2	176.46 MHz	176.46 MHz	clk	

Figure 6: Fmax

Resource utilisation:

Flow Status	EDA Netlist Writer Failed - Sun Apr 12 13:02:18 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	FFT
Top-level Entity Name	FIR_IP_16
Family	Cyclone V
Device	5CGXFC9E7F35C8
Timing Models	Final
Logic utilization (in ALMs)	1,072 / 113,560 (< 1 %)
Total registers	2258
Total pins	53 / 616 (9 %)
Total virtual pins	0
Total block memory bits	4,970 / 12,492,800 (< 1 %)
Total DSP Blocks	6 / 342 (2 %)
Total HSSI RX PCSs	0 / 12 (0 %)
Total HSSI PMA RX Deserializers	0 / 12 (0 %)
Total HSSI TX PCSs	0 / 12 (0 %)
Total HSSI PMA TX Serializers	0 / 12 (0 %)
Total PLLs	0 / 20 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 7: Flow summary

- With length of FFT being 8, but variable streaming input Timing analysis

Slow 1100mV 85C Model Fmax Summary				
 <<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	97.27 MHz	97.27 MHz	altera...ed_tck	
2	184.23 MHz	184.23 MHz	clk	

Figure 8: Fmax

Resource utilisation:

Flow Status	EDA Netlist Writer Failed - Sun Apr 12 12:31:10 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	FFT
Top-level Entity Name	FFT_IP
Family	Cyclone V
Device	5CGXFC9E7F35C8
Timing Models	Final
Logic utilization (in ALMs)	573 / 113,560 (< 1 %)
Total registers	1190
Total pins	57 / 616 (9 %)
Total virtual pins	0
Total block memory bits	216 / 12,492,800 (< 1 %)
Total DSP Blocks	2 / 342 (< 1 %)
Total HSSI RX PCSs	0 / 12 (0 %)
Total HSSI PMA RX Deserializers	0 / 12 (0 %)
Total HSSI TX PCSs	0 / 12 (0 %)
Total HSSI PMA TX Serializers	0 / 12 (0 %)
Total PLLs	0 / 20 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 9: Flow summary

14 Conclusion

The pipelined streaming FFT architecture successfully computes the 8-point FFT with reduced latency and improved throughput. Results were verified against MATLAB and Intel FFT IP.